

Kafka 面试题整理

适用场景：后端开发、Java/Spring Boot、消息队列、高并发、异步系统设计面试。

使用建议：这份资料按“概念 -> 机制 -> 生产实践 -> 项目落地”来整理。真正面试时不要逐题背答案，建议先记住结论，再用 2 到 3 句话展开。

一、基础概念

1. Kafka 是什么？适合解决什么问题？

Kafka 是一个分布式消息队列和流式数据平台，核心能力是高吞吐、可持久化、可水平扩展。它适合做异步解耦、削峰填谷、日志采集、埋点传输、事件驱动和实时数据管道。

2. Topic、Partition、Replica 分别是什么？

Topic 是逻辑上的消息主题。Partition 是 Topic 的物理分片，Kafka 通过分区来提升并发和吞吐。

Replica 是分区副本，用来保证高可用；一个分区会有一个 Leader，其他副本是 Follower。

3. Kafka 为什么吞吐量高？

原因主要有四点：顺序写磁盘、零拷贝、批量发送/批量拉取、分区并行。Kafka 不追求单条消息的低延迟极致，而是通过顺序 I/O 和批处理把整体吞吐做上去。

4. Kafka 是推模型还是拉模型？

消费者本质上是拉模型，由 Consumer 主动 poll 数据。这样做的好处是消费速率由客户端自己控制，既方便做批量拉取，也更容易应对慢消费者。

二、Producer 相关

5. Producer 发送消息时怎么决定发到哪个分区？

如果显式指定 partition，就直接发到指定分区。如果指定了 key 但没指定 partition，通常会按 key 做哈希，相同 key 尽量落到同一分区。如果 key 也没有，默认分区器会做轮询或粘性分配。

6. `acks` 有哪些取值？分别代表什么？

`acks=0` 表示 Producer 发完就不等 Broker 确认，性能高但容易丢。`acks=1` 表示 Leader 写入成功就返回，Follower 还没同步时仍有丢失风险。`acks=all` 或 `-1` 表示 ISR 中满足条件的副本确认后才返回，可靠性最高。

7. Kafka 怎么尽量保证消息不丢？

生产端一般会配 `acks=all`、重试、合理的 `retries` 和 `delivery.timeout.ms`。Broker 侧要保证副本数、ISR 和刷盘策略合理。消费端不能“先提交 offset 再处理业务”，否则消息还没真正落业务侧就被认为消费成功了。

8. 什么是幂等生产者？

幂等生产者是 Kafka 为 Producer 提供的去重能力，开启 `enable.idempotence=true` 后，Broker 会结合 ProducerId 和序列号识别重复消息，减少因为重试导致的重复写入。它主要解决单分区、单会话内的重复发送问题。

9. 幂等生产者和事务有什么区别？

幂等生产者主要解决“重试导致重复写”的问题。事务是在此基础上进一步保证多分区、多主题写入以及“消费 + 生产”链路的原子性。面试里可以概括成：幂等解决重复，事务解决原子提交。

三、Consumer 与 Offset

10. Consumer Group 是什么？

同一个 Consumer Group 内，一个分区在同一时刻只会被其中一个消费者实例消费，所以 Kafka 的并行消费能力上限通常受分区数限制。多个 Group 可以各自独立消费同一个 Topic。

11. Offset 是什么？存在哪里？

Offset 是消费者在某个分区上的消费位点，可以理解成“这组消费者处理到哪了”。现代 Kafka 默认把消费位点存到内部 Topic `__consumer_offsets` 中，而不是存到 ZooKeeper。

12. 自动提交 offset 和手动提交 offset 有什么区别？

自动提交简单，但可能出现“消息还没真正处理成功，位点已经提交”的问题。手动提交更适合和业务处理结果绑定，能把“处理成功后再提交”的语义做清楚，代价是代码复杂度更高，也更容易因为漏提交造成重复消费或积压。

13. 为什么手动提交 offset 时容易出现消费停滞？

因为位点前进是靠代码显式提交的。如果业务异常后没有执行 `ack.acknowledge()` 或 `commitSync/commitAsync`，同一个 offset 会反复被拉取，lag 看起来就像卡住了。这个问题本质上不是 Kafka 不消费，而是消费者一直卡在同一条消息。

14. 手动提交 offset 导致消费停滞时，你会怎么排查？

先看 Consumer Group 的 lag、当前 offset 和日志末尾 offset。再看消费者日志里是否反复打印同一个 topic-partition-offset。然后检查代码是否吞异常、是否漏掉 ack、是否存在毒消息反复失败。生产上通常会补日志、重试和死信 Topic，避免一条坏消息长期卡住整个分区。

15. `commitSync` 和 `commitAsync` 怎么选？

`commitSync` 成功语义更明确，但阻塞更强。`commitAsync` 吞吐更好，但失败处理更复杂。面试里一般回答：对一致性要求更高的关键链路更偏向同步提交；追求吞吐时可以异步提交，但要注意回调里做失败处理。

16. 什么情况下会触发 Rebalance？

常见场景有：消费者实例数量变化、订阅主题变化、分区数变化、消费者心跳超时。Rebalance 期间分区会重新分配，如果处理不当容易造成短暂停顿、重复消费或缓存抖动。

17. 怎么减轻 Rebalance 带来的影响？

可以用 Cooperative Rebalance、稳定的分区数规划、合理设置 `session.timeout.ms` 和 `max.poll.interval.ms`，还要避免业务处理过慢导致长时间不 poll。核心思想是减少频繁成员变动，并让单次处理时间可控。

四、存储与高可用

18. Leader、Follower、ISR 分别是什么？

Leader 负责对外提供读写。Follower 负责从 Leader 同步数据。ISR 是“与 Leader 保持足够同步的副本集合”，只有 ISR 里的副本才参与高可靠确认。Kafka 的高可用不是看所有副本，而是重点看 ISR 状态。

19. 什么是 HW 和 LEO？

LEO 是日志末尾位点，表示副本当前写到了哪里。HW 是高水位，表示对消费者可见的最大安全位点。消费者通常只能读到 HW 之前的数据，这样能避免读到还没完成副本同步的不安全消息。

20. Kafka 是怎么做持久化的？

Kafka 会把消息顺序追加到磁盘日志段文件里，并按 segment 管理。旧 segment 会根据保留策略删除或压缩。因为是顺序写，所以即使落磁盘，整体吞吐依然很高。

21. 保留策略 `retention` 和日志压缩 `log compaction` 有什么区别？

`retention` 是按时间或大小删除旧消息。`log compaction` 是按 key 保留最新值，更适合配置同步、状态流或变更日志场景。前者关注“留多久”，后者关注“每个 key 留什么”。

五、顺序性、重复、事务

22. Kafka 能保证全局有序吗？

不能天然保证全局有序，只能保证单个分区内的顺序。如果业务真的要求顺序，通常会让同一业务 key 路由到同一分区，再在消费端按分区顺序处理。

23. Kafka 会不会重复消费？

会。只要消费者在“处理成功”与“提交 offset”之间崩溃，重启后就可能重新处理之前的消息。所以面试里不要说 Kafka 天然 exactly once，应该说默认语义更接近 at-least-once，业务侧需要幂等兜底。

24. 怎么做消费幂等？

常见方式有三类：业务唯一键去重、数据库唯一约束、Redis 幂等键。例如把订单号、事件 ID、业务流水号当作幂等键，处理前先查是否已经消费过。幂等是 Kafka 项目里最常见、最实用的防重方案。

25. Kafka 事务解决了什么问题？

事务主要解决“写多分区、多 Topic，或者消费后再生产时，如何做到原子提交”的问题。它能配合幂等生产者提供 EOS 语义，但使用成本更高，对性能和系统复杂度都有影响。

26. 你怎么理解 Exactly Once？

严格讲，Exactly Once 不是只靠一个配置项就自动获得的。Broker、Producer、Consumer、下游存储都要协同设计。很多业务面试里更务实的答案是：链路内部尽量做到 EOS，落业务库时仍然要用幂等保障最终正确性。

六、生产实践与排障

27. 如何排查消息积压？

先看 Topic 各分区 lag、生产速率、消费速率，再判断是“生产太快”还是“消费太慢”。如果消费慢，要继续查是业务处理耗时、下游依赖卡住、单分区热点、rebalance 频繁，还是消费者实例数和分区数不匹配。

28. Kafka 支持死信队列吗？

Kafka Broker 本身没有像 RabbitMQ 那样的原生死信队列机制，但工程里通常会用 Dead Letter Topic 作为死信 Topic。也就是消费失败达到阈值后，由应用把消息转发到另一个 Topic，再让主消费位点继续前进。

29. 为什么死信 Topic 能避免分区卡死？

如果一条消息一直处理失败而又不提交 offset，同一个分区就会反复卡在这条消息上。把它转存到 DLT 之后，主消费者就可以提交当前位点并继续消费后面的消息，从而避免整个分区长期停滞。

30. 什么时候不适合直接跳过坏消息？

当业务要求严格顺序、每条消息都必须落地，或者坏消息本身可能代表核心数据丢失时，就不能轻易旁路。此时更适合暂停消费、人工介入、修复数据后再继续，而不是简单丢到 DLT 就往后走。

31. 常见监控指标有哪些？

重点看生产端发送失败率、重试率、请求延迟；Broker 侧看吞吐、ISR 变化、请求耗时、磁盘和网络；消费端看 lag、rebalance 次数、poll 间隔、处理耗时、死信量。监控一定要结合告警阈值，而不是只看 dashboard。

七、结合项目的回答模板

32. 项目里为什么会关闭自动提交，改用手动提交？

因为我们希望把“消息处理成功”和“位点提交成功”绑定在一起，避免业务没执行完 offset 就前进。在 Spring Kafka 里通常会配置 `enable-auto-commit=false`，再用 `ack-mode>manual` 或 `manual_immediate` 控制提交时机。

33. 如果面试官问：手动提交 offset 时，消费者没正确提交怎么办？

可以回答：先看 lag 是否持续增长，再确认是否反复消费同一 offset，然后排查业务异常是否被吞掉、ack 是否漏调、是否存在毒消息。落地方案一般是补充日志、限制重试次数、失败后转发 DLT，并在合适时机提交 offset。

34. 如果面试官问：Kafka 和 RabbitMQ 怎么选？

如果更关注高吞吐、日志流、埋点、事件流和大规模分区扩展，Kafka 更合适。如果更关注复杂路由、延迟队列、优先级、原生死信等传统 MQ 能力，RabbitMQ 更方便。没有绝对优劣，关键看业务模型。

35. 如果面试官问：Kafka 和 RocketMQ 怎么选？

可以回答：Kafka 在生态、流式处理、日志管道方面更成熟；RocketMQ 在国内业务场景、顺序消息、事务消息和运维体验上也很强。真正选型通常看团队经验、现有基础设施和业务诉求，而不是只看单项性能。

八、快速背诵版

36. 一分钟总结怎么答？

Kafka 是一个高吞吐、可持久化、可扩展的分布式消息系统，核心设计是 Topic 分区、副本和消费者组。它通过顺序写、批处理和分区并行提升吞吐，通过 ISR 和副本机制保证高可用。生产端要关注 `acks`、重试、幂等和事务；消费端要关注 offset 提交、重复消费、rebalance 和 lag。真正落项目时，关键不是背概念，而是把“顺序、可靠、幂等、监控、排障”这五件事讲清楚。

37. 面试最后的加分句怎么说？

“我对 Kafka 的理解不会停留在概念上，真正上线时我最关注的是三件事：消息会不会丢、会不会重复、积压时怎么恢复。很多线上问题最后都不是 Kafka 本身挂了，而是 offset 提交、幂等设计和异常处理没做好。”